Lógica secuencial, clock, flip-flops y registros de desplazamiento

Resumen de estudio de los videos #48 a #54 y #56 del curso **"Arquitectura y Lógica de Computadoras"** del canal Hacker Lykeion.

#	Video	Tema	Sección
48	Intro a Lógica Secuencial	Qué es la lógica secuencial, realimentación	1
49	Circuitos Secuenciales: ¿qué es el clock?	Sincrónico vs. asincrónico, clock, tipos de disparo	2 y 3
50	Intro a los Flip Flop	Qué es un flip-flop, familias, símbolos	4
51	Flip Flop SR	SR, entradas sincrónicas y asincrónicas	5 y 9
52	Flip Flop D	D	6
53	Flip Flop T	T	7
54	Flip Flop JK	JK	8
56	Registros de desplazamiento	Serie/paralelo, SIPO, SISO, PIPO, PISO	11 y 12

Notación usada en todo el resumen

- `Q` = valor de la salida en el **presente**.
- `Q*` = valor de la salida en el **futuro** (después del próximo disparo de clock).
- $\bar{Q}$ = salida negada (lo contrario de `Q`).
- `X` = "no importa" (vale lo mismo con 0 o con 1).
- Las notas marcadas con ⚠ señalan lapsus del video o agregados que no están en el material original.

1. Introducción a la lógica secuencial

1.1 Definición

- En la **lógica combinacional** (lo visto antes en el curso), la salida depende **solo de las entradas** de ese momento.
- En la **lógica secuencial**, la salida depende de:
  - las **entradas** actuales, y
  - el **valor actual de las salidas** (el estado presente).
- Dicho de otra forma: **el estado futuro de la salida es función de las entradas y de las salidas del presente.**

1.2 Cómo se logra: la realimentación

- Un circuito secuencial es una interconexión de compuertas (funciones lógicas) en la que **algunas salidas vuelven a entrar como entradas adicionales**.
- Ejemplo genérico del video: un bloque con 3 entradas y 3 salidas, donde 2 de las salidas se reingresan como 2 entradas nuevas. El circuito "ve" entonces 5 líneas: las 3 entradas y las 2 salidas actuales. Con esa combinación genera la nueva combinación de salidas.

1.3 Ejemplo: compuerta OR realimentada

El ejemplo no es útil en la práctica, pero sirve para entender la idea de **estado**.

- Se toma una compuerta OR de dos entradas.
- Una entrada es `A`. La otra entrada se conecta a la propia salida `Z`.
- Expresión: **`Z* = A + Z`** (la Z del futuro es A OR la Z del presente).

Paso a paso:

Paso	A	Z (presente)	Z* (futuro)	Qué pasa
1. Estado inicial	0	0	0	Queda en 0 indefinidamente (hasta que se desenchufe).
2. Se pone A = 1	1	0 o 1	1	La salida pasa a 1 sin importar lo que valía Z.
3. Se vuelve A = 0	0	1	1	El 1 de la salida vuelve a entrar por la realimentación: sigue en 1.

Conclusiones del ejemplo:

- Una vez que `A` valió 1, el circuito queda **"trabado" en 1 para siempre**: recuerda que alguna vez hubo un 1.
- Tal como está, **no hay forma de sacarlo de ese estado**.
- Solución sugerida: poner otra compuerta en la rama de realimentación que permita **resetearlo** (volverlo a 0). Esa idea es la que lleva a los flip-flops.

2. Circuitos secuenciales asincrónicos y sincrónicos

Hay dos tipos de circuitos lógicos secuenciales. En ambos, las salidas del futuro son función de las entradas y de las salidas del presente.

	Asincrónico	Sincrónico
Entrada de clock	No tiene	Tiene una entrada adicional: **clock** o **sincronismo**
Cuándo cambia de estado	Apenas cambian las entradas	Solo cuando la señal de clock lo **dispara** (lo acciona)

- La señal de clock **le marca el tiempo** a los circuitos sincrónicos: les indica el momento exacto en que deben pasar al nuevo estado.

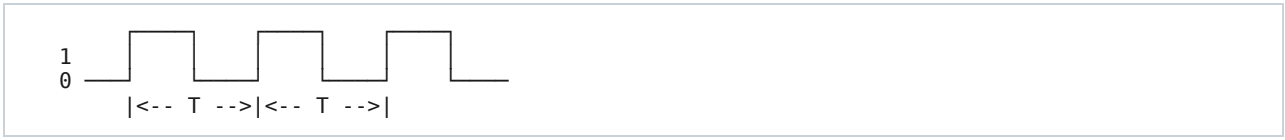
3. El clock

3.1 Qué es

- Es una señal generada por una **fuente de clock**: un dispositivo electrónico que produce una señal de sincronismo.
- La señal alterna entre dos valores:
  - **nivel alto** = 1 lógico,
  - **nivel bajo** = 0 lógico.
- Está un tiempo en 1, un tiempo en 0, y se repite sin detenerse: detrás de un pulso siempre viene otro.
- Idealmente:
  - el tiempo en 1 y el tiempo en 0 son **exactamente iguales**;
  - esos tiempos **nunca varían** (la señal es lo más estable posible).
- En un simulador se ve a escala humana. En una computadora real ocurre muchísimo más rápido.

3.2 El clock como función matemática periódica

- En un diagrama temporal, el eje X es el **tiempo** (en segundos) y el eje Y es la **amplitud** (por ejemplo, en voltios).
- El clock es una **función periódica**: toma un conjunto de valores (un 1 seguido de un 0) y ese conjunto se repite una y otra vez, indefinidamente.



Período (T)

- Es el **tiempo que dura** el conjunto de valores que se repite (un 1 más un 0).
- Se mide en **segundos**.
- Todos los períodos de un mismo clock son iguales.

Frecuencia (f)

- Es la **cantidad de veces que la señal se repite en 1 segundo**.
- Es la inversa del período: `f = 1 / T`.
- Su unidad es 1/segundo, que se llama **hertz (Hz)**. 1 Hz = algo que se repite una vez por segundo.
- Analogía del video: "frecuencia" es cuántas veces ocurre algo por unidad de tiempo (lavarse los dientes 2 veces por día, cumplir años 1 vez por año). Para las señales, la unidad de tiempo de referencia es el segundo.
- Múltiplos usados para clocks de microprocesadores: **Hz, kHz, MHz y GHz**.

Velocidad de clock

- La velocidad de un circuito secuencial se expresa por su **frecuencia de clock** (más correcto que decir "velocidad de clock").
- Indica **cuán rápido el circuito pasa de un estado a otro**.

3.3 Cómo comparar clocks en un diagrama temporal

En el diagrama se ve **tiempo**, no repeticiones por segundo. Por eso:

- Pulsos **anchos** (mucho tiempo en 1 y mucho en 0) → período largo → frecuencia baja → clock **más lento**.
- Pulsos **angostos** → período corto → frecuencia alta → clock **más rápido**.

Ejemplo del video con tres señales: la **roja** (pulsos más anchos) es la más lenta, la **verde** (pulsos más angostos) es la más rápida y la **azul** es intermedia.

3.4 Clock y velocidad del microprocesador

- Clock más lento → los pasos internos se hacen más lento → la computadora habitualmente procesa más lento.
- Clock más rápido → los pasos internos se hacen más rápido → procesa más rápido.

Pero comparar dos microprocesadores distintos solo por su frecuencia no es válido.

- Ejemplo: uno de 2 GHz y otro de 3 GHz. El de 3 GHz **no es necesariamente más rápido**.
- El de menor frecuencia puede tener hardware específico que le dé un rendimiento comparable o superior.
- Hay que distinguir dos cosas:
  - la velocidad de los **procesos internos** (lo que marca el clock);
  - las **instrucciones por segundo** que realmente ejecuta.
- Esa comparación directa se podía hacer en los **años 90**. Desde los **2000** en adelante ya no es válida, por cómo funcionan los microprocesadores.
- Dato: hace unos 20 años que las frecuencias se mantienen por debajo de los 5 o 6 GHz.

3.5 Ver el clock en una computadora con GNU/Linux

Procedimiento mostrado en el video:

1. `cat /proc/cpuinfo`
   - `cpuinfo` es un archivo de la carpeta `/proc` con información del microprocesador.
   - `cat` muestra el contenido de archivos (o los concatena).

- El dato que interesa es **`cpu MHz`**: la frecuencia de clock en el instante en que se leyó el archivo. En el ejemplo: 1228 MHz (≈ 1,2 GHz).

2. `cat /proc/cpuinfo | grep MHz`
   - El pipe (`|`) pasa la salida a `grep`, que filtra las líneas que contienen "MHz".
   - Aparece una línea por cada núcleo o hilo (thread) del microprocesador.
3. Usar el programa `watch` para repetir el comando automáticamente una vez por segundo y ver una "foto" de las frecuencias cada segundo.
   - ⚠ El video no deletrea el comando completo. Una forma equivalente es `watch -n 1 "grep MHz /proc/cpuinfo"`.

Lo que se observa (micro con frecuencia **nominal de 2,5 GHz**):

- Cada núcleo tiene su propia frecuencia y cambia todo el tiempo: algunos a 2700 MHz, otros a 600 MHz.
- En modo **ahorro de energía**: las frecuencias de los núcleos bajan. La computadora reacciona y procesa más lento (se nota en tareas intensivas como renderizar un video o comprimir un archivo).
- En modo **alto rendimiento**: todos los núcleos suben (por encima de 2000 MHz).
- Al darle una tarea (por ejemplo, reproducir un video), responde mejor cuanto más alto es el clock.

3.6 Clock, energía y calor

	Clock más rápido	**Clock más lento**
Procesamiento	Más rápido	Más lento
Consumo de energía	Mayor (el micro y los componentes asociados)	Menor
Calor generado	Mayor	Menor
Batería (notebook)	Dura menos	Dura más

- El problema del clock alto no es solo el gasto de electricidad: **más energía implica más calor**.
- Si la computadora no está bien refrigerada, el calor se acumula y:
  - **acorta la vida útil** de los componentes, o
  - los **quema** (daño irreparable).
- Por eso es habitual **bajar la frecuencia cuando no hay una tarea intensiva**. Lo regulan automáticamente el **sistema operativo y el microprocesador**.

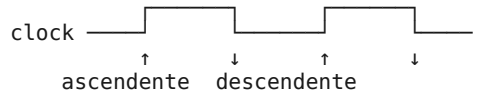
Overclocking

- Algunas placas madre (motherboards) permiten **forzar un valor de clock** para el microprocesador y otros componentes internos. Otras no.
- Si se quiere llevar el micro al máximo, o exigirlo **más allá del máximo seguro indicado por el fabricante**, hay que verificar antes de comprar que la placa madre soporte modificar arbitrariamente la frecuencia de clock.

3.7 Tipos de disparo de clock

El clock es una sola señal, pero los dispositivos pueden accionarse con **distintas partes** de ella. Hay dos grandes tipos, cada uno con dos variantes:

Tipo	Variante	Cuándo se dispara el dispositivo
Por nivel	Nivel alto	Mientras la señal está en 1. Se puede pensar como un "piquito" breve hacia arriba.
	Nivel bajo	Mientras la señal está en 0. La señal está normalmente en alto y baja un instante: un "vallecito".
Por flanco (transición)	Flanco ascendente	En el instante en que la señal pasa de 0 a 1.
	Flanco descendente	En el instante en que la señal pasa de 1 a 0.



Detalles del disparo por flanco:

- El disparo ocurre **en la transición, no en el nivel**. Es un instante "infinitamente chiquito".
- Una vez que la señal quedó en alto, no pasa nada más. Tiene que bajar y volver a subir para producir el siguiente disparo (en el caso ascendente).
- Los clocks por flanco son los **preferibles**: el disparo es **más preciso** y se comportan **mejor frente al ruido**.

⚠ En el resumen final del video se dice "si el disparo es por flanco ascendente, será cuando el flanco baja". Es un lapsus: el que dispara cuando el flanco baja es el **descendente**.

3.8 El clock real

- En manuales y en instrumentos de medición el clock se ve perfecto, con flancos verticales.
- En realidad, haciendo zoom en el tiempo, los flancos son **rampas** de pendiente muy pronunciada.
- **No existe una transición en tiempo cero** entre alto y bajo. Eso es solo "de papel" o académico.
- Es un problema para los circuitos disparados por flanco, que necesitan una **transición abrupta**.
- Solución: existen **circuitos especiales** que hacen más abruptos los flancos.

4. Flip-flops: conceptos generales

4.1 Definición

- El flip-flop es el **bloque de construcción fundamental** de los circuitos lógicos secuenciales.
- Es una **memoria de un bit**: puede almacenar un 0 o un 1.
- Mantiene el bit **mientras tenga alimentación eléctrica**. Si se queda sin energía, pierde la información.

4.2 Clasificación

Según el clock

- **Sincrónicos**: tienen entrada de clock. Son los que se estudian en el curso.
- **Asincrónicos**: mismas entradas y salidas, pero sin entrada de clock.

Según la familia

- **SR**, **D**, **T** y **JK**.

Según el tipo de disparo (solo los sincrónicos): por nivel o por flanco (ver 3.7).

4.3 Cómo leer la entrada de clock en un diagrama en bloques

Símbolo en la entrada de clock	Tipo de disparo
Entrada común (solo dice "clock")	Nivel **alto**
Entrada con **circulito**	Nivel **bajo**
Entrada con **triangulito** (o ángulo)	Flanco **ascendente**
Circulito + triangulito	Flanco **descendente**

Regla práctica: el triangulito significa "flanco" y el circulito significa "invertido" (bajo o descendente). Esto vale para cualquier circuito secuencial sincrónico, no solo para los flip-flops.

4.4 Características comunes a todos los flip-flops

- **Dos salidas: `Q` y `Q̄`**, donde una es la inversa de la otra.
  - Los circuitos de los flip-flops son simétricos (la mitad de arriba es un espejo de la de abajo).
  - `Q̄` no se generó con un propósito específico: es **inherente a cómo está construido** el circuito.
- **El clock no aparece en la tabla de verdad.** El clock solo indica el momento del disparo; no describe el comportamiento. Hay que leer cada fila como "cuando llega el disparo de clock, la salida pasa a valer Q*". (Si se quisiera construir el flip-flop con compuertas, ahí sí habría que incluir el clock en la tabla.)
- **Presente y futuro**: cada pulso de clock separa dos momentos adyacentes. Al llegar el disparo, `Q` toma el valor de `Q*`; ese valor pasa a ser el nuevo presente, y en el pulso siguiente habrá un nuevo `Q*`.
- **"Nuevo estado" no significa "estado distinto"**: el nuevo estado puede ser igual al anterior. Si `Q = 0` y `Q* = 0`, el flip-flop está memorizando.
- **Lo que ocurre entre dos disparos de clock es invisible para el flip-flop.** Solo importan los valores que tienen las entradas en el momento del disparo.
- Pueden tener, además, entradas asincrónicas **preset** y **clear** (ver sección 9).

4.5 Relación entre las familias y usos

- El **SR no se usa como tal** en las computadoras: forma parte de los demás. Agregando compuertas a un SR se obtiene un D, un T o un JK.
- En general, teniendo **cualquiera de los cuatro** se lo puede transformar en cualquiera de los otros tres agregando compuertas.
- Usos de D, T y JK:
  - **Registros**: almacenar bits de información de forma transitoria.
  - **Contadores**: contar pulsos de clock o alguna secuencia que ocurre dentro de la computadora.

5. Flip-flop SR (Set-Reset)

5.1 Entradas y salidas

- Entradas: **S** (set) y **R** (reset).
- Entrada de clock (en el video, disparado por flanco ascendente).
- Salidas: `Q` y `Q̄`.

5.2 Tabla de verdad simplificada

S	R	Q*	Estado
0	0	Q	**Memoria**: conserva el valor que tenía
0	1	0	**Reset**: fuerza un 0
1	0	1	**Set**: fuerza un 1
1	1	—	**Indeterminado**: no se puede usar

5.3 Tabla de verdad completa

S	R	Q	Q*	Estado
0	0	0	0	Memoria
0	0	1	1	Memoria
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	X	—	No se usa

Explicación:

- **Set** (`S=1, R=0`): al llegar el pulso de clock, la salida queda en 1 **sin importar** el valor anterior.
- **Reset** (`S=0, R=1`): al llegar el pulso de clock, la salida queda en 0 **sin importar** el valor anterior.
- **Memoria** (`S=0, R=0`): el futuro es igual al presente. Eso es justamente memorizar el bit.
- **`S=1, R=1`**: estado indeterminado. Por este motivo el SR no se usa directamente y se le agregan compuertas para obtener otra tabla (D, JK).

5.4 Ejemplo en el simulador (diagrama temporal)

Estado inicial: `S=0`, `R=0`, `Q=1`.

1. Se pone `R=1` → llega un disparo de clock → `Q` pasa a **0**.
2. Se vuelve `R=0` → con `S=0` y `R=0`, los siguientes pulsos **memorizan**: `Q` sigue en 0.
3. Se pone `S=1` → llega un disparo de clock → `Q` pasa a **1**.
4. Se vuelve `S=0` → `Q` **sigue en 1** (memoria).

Conclusión: no importa que después se "quite" S o R. Las entradas S y R se usan para forzar la salida a 1 o a 0 **en el siguiente instante de clock**; luego, con ambas en 0, el valor queda guardado.

6. Flip-flop D

6.1 Entradas y salidas

- **Una sola entrada: D** (el dato).
- Entrada de clock.
- Salidas `Q` y $\bar{Q}$.
- Puede tener o no preset y clear.

6.2 Tabla de verdad

D	Q*
0	0
1	1

6.3 Cómo entenderlo

- Parece una tabla sin sentido, pero la clave está en el **momento**: cuando llega el disparo de clock, `Q` **toma el valor de D y lo memoriza**.
- Aunque después `D` cambie, `Q` queda fijo hasta el próximo disparo.
- Analogía: es como **sacar una foto** del bit que hay en `D` y guardarla en `Q`. Cada disparo de clock saca una foto nueva.

6.4 Ejemplo en el simulador (D disparado por flanco ascendente)

- `D` pasa a 0 → en el siguiente flanco ascendente, `Q` pasa de 1 a 0.
- `D` pasa a 1 → en el siguiente flanco ascendente, `Q` pasa de 0 a 1.
- Se genera un pulso de `D` **muy angosto**, que empieza y termina **antes** del flanco ascendente → `Q` **no cambia**. El flip-flop nunca lo vio.

6.5 Para qué se usa

- Para que `Q` **no se vea afectada** por los cambios de `D` mientras no haya disparo de clock.
- Útil para **manejar buses**.
- Útil en dispositivos que toman valores y los guardan un tiempo hasta poder procesarlos: funciona como una memoria o **buffer**.

7. Flip-flop T

7.1 Entradas y salidas

- Una entrada: **T**.
- Entrada de clock.
- Salidas `Q` y $\bar{Q}$.
- Preset y clear (habitualmente **activos en bajo**).

7.2 Tabla de verdad simplificada

T	Q*	Comportamiento
0	Q	**Memoria**
1	$\bar{Q}$	**Niega** lo que tenía memorizado

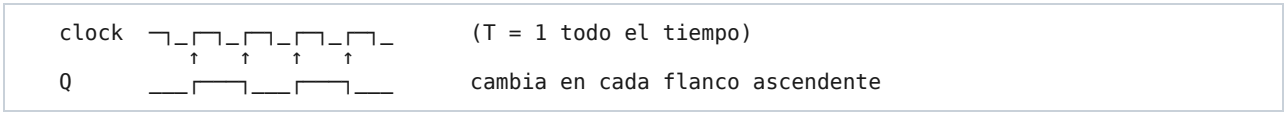
7.3 Tabla de verdad completa

T	Q	Q*	Comportamiento
0	0	0	Memoriza
0	1	1	Memoriza
1	0	1	Invierte
1	1	0	Invierte

7.4 Ejemplo en el simulador

- Con `T=1` **sostenido**: en **cada flanco ascendente** del clock, `Q` cambia de estado (0 → 1 → 0 → 1 …).
  - La salida se ve como una copia del clock **"estirada" en el tiempo**: como `Q` cambia una vez por ciclo de clock, un ciclo completo de `Q` (un 1 y un 0) tarda **dos ciclos de clock**.
- Se pone `T=0` → `Q` queda **memorizando** el último valor.
- Se pone `T=1` justo antes de un flanco ascendente → `Q` se invierte. Si se quita el 1 de `T` **antes** del siguiente flanco → `Q` queda fija en ese nuevo valor.

Regla: **T en 0 memoriza; T en 1 niega el estado memorizado.** El cambio siempre ocurre cuando llega el pulso de clock.



8. Flip-flop JK

8.1 Entradas y salidas

- Entradas: **J** y **K**.
- Entrada de clock.
- Salidas `Q` y $\bar{Q}$.
- Puede tener preset y clear.

Idea central: el JK **incluye el comportamiento de los flip-flops anteriores** (SR y T).

⚠ En el video se dice "son siempre asincrónicos los que vemos". Es un lapsus: tienen entrada de clock, o sea que son **sincrónicos**.

8.2 Tabla de verdad simplificada

J	K	Q*	Estado	Equivale a…
0	0	Q	**Memoria**	SR con S=0 y R=0, o T con T=0
0	1	0	**Reset** (fuerza 0)	SR con R=1
1	0	1	**Set** (fuerza 1)	SR con S=1
1	1	$\bar{Q}$	**Memoria negada** (invierte)	T con T=1

- `J` cumple el papel de `S` (set) y `K` el de `R` (reset).
- La diferencia con el SR: la combinación `1 1`, que en el SR era indeterminada, en el JK **sí se usa** e invierte la salida.

8.3 Tabla de verdad completa

J	K	Q	Q*	Estado
0	0	0	0	Memoria
0	0	1	1	Memoria
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	1	Memoria negada
1	1	1	0	Memoria negada

8.4 Ejemplo en el simulador

1. `J=1`, `K=0` → en el flanco ascendente, `Q` pasa a **1** (set).
2. `J=0`, `K=0` → `Q` queda **memorizada**.
3. `J=0`, `K=1` → en el siguiente flanco ascendente, `Q` pasa de 1 a **0** (reset). Si llegan más pulsos con `K=1`, `Q` sigue en 0.
4. `J=0`, `K=0` → `Q` queda memorizada en 0. Cada pulso de clock sigue memorizando el estado anterior.
5. `J=1`, `K=1` → **cada flanco ascendente niega** el estado anterior de `Q`. Se comporta como un flip-flop T.
6. Se deja que la salida llegue a 1 y se vuelve a `J=0`, `K=0` → los pulsos siguientes memorizan ese 1.

⚠ En el paso 1 el relato del video dice "la salida Q se pone en cero", y en el paso 3 dice "porque J vale uno". Son lapsus al hablar: según la tabla del propio video, `J=1` fuerza un **1** y el que fuerza el 0 es **`K=1`**.

9. Entradas sincrónicas y asincrónicas (preset y clear)

9.1 Entradas sincrónicas

- Son las entradas de datos: **S, R, D, T, J, K**.
- **Solo tienen efecto cuando llega un disparo de clock.**
- Demostración del video: con el clock deshabilitado, se mueven `S` y `R` y a `Q` no le pasa nada.

9.2 Entradas asincrónicas

- **Clear**: fuerza `Q = 0`.
- **Preset**: fuerza `Q = 1`.
- Actúan **sin esperar al clock**.
- Habitualmente son **activas en bajo**.
- No todos los flip-flops las tienen, pero es común.

Para qué sirven:

- Al **energizar** el dispositivo (conectar la batería, enchufarlo o encenderlo), permiten, con un circuito electrónico asociado, forzar un **estado inicial conocido**: 0 con clear, 1 con preset.
- También se pueden usar durante el funcionamiento para forzar un estado específico.
- Ejemplo de uso: un **contador**, que necesita empezar desde un estado determinado.

10. Cuadro comparativo de los flip-flops

Flip-flop	Entradas	Comportamiento (al llegar el clock)	Particularidad	Uso
SR	S, R	`00` memoria · `01` reset · `10` set	`11` es indeterminado	No se usa solo; es la base de los demás
D	D	`Q* = D`	"Saca una foto" de D	Registros, buses, buffers
T	T	`0` memoria · `1` invierte	Con T=1, Q cambia en cada disparo	Contadores
JK	J, K	`00` memoria · `01` reset · `10` set · `11` invierte	Reúne el SR y el T, sin estado prohibido	Registros, contadores

⚠ **Complemento (no está en los videos):** las tablas anteriores se pueden resumir en ecuaciones, útiles para ejercicios.

- SR: $Q^* = S + \bar{R} \cdot Q$ (con la condición $S \cdot R = 0$)
- D: $Q^* = D$
- T: $Q^* = T \oplus Q$
- JK: $Q^* = J \cdot \bar{Q} + \bar{K} \cdot Q$

11. Comunicación serie y paralelo

A nivel de bits, las comunicaciones se clasifican según si los bits viajan **uno detrás de otro** o **todos a la vez**.

	Serie	Paralelo
Cómo viajan los bits	Uno detrás de otro, por la misma línea	Todos simultáneamente, cada uno por su propio cable
Cables para 8 bits	1 línea de datos + 1 cable de masa	8 líneas de datos + 1 cable de masa
Tiempo para 8 bits	8 "turnos"	1 solo "turno"

- El cable de **masa** (tierra, común, negativo o referencia de 0 V) está en ambos casos.
- Por la línea de datos se envían **pulsos de tensión** que representan un 0 o un 1.
- Ejemplo del video: los 8 bits `0 1 1 1 0 0 1 1`. En serie se mandan uno tras otro; en paralelo, cada bit va por un cable distinto al mismo tiempo.

12. Registros de desplazamiento

12.1 Definición

- Un **registro** es un **conjunto de flip-flops** que almacena bits con algún propósito.
- Todos se construyen con flip-flops y tienen entrada de **clock**.
- Se diferencian en **cómo entran** y **cómo salen** los datos.

12.2 Clasificación

Hay dos posibilidades para la entrada y dos para la salida, por lo tanto **cuatro tipos**:

	Salida serie (SO)	Salida paralelo (PO)
Entrada serie (SI)	**SISO**	**SIPO**
Entrada paralelo (PI)	**PISO**	**PIPO**

Significado de las siglas: **S**erial / **P**arallel, **I**n / **O**ut.

12.3 Para qué sirve cada uno

PISO (entrada paralelo, salida serie): serializar

- Los bits se cargan todos juntos (puede haber un pin adicional que indica la carga). Luego, cada pulso de clock saca un bit por la salida serie.
- Ejemplo: un microprocesador con un bus de datos paralelo de 64 bits que quiere transmitir a otro dispositivo sin usar 64 cables.

SIPO (entrada serie, salida paralelo): paralelizar

- Es el inverso del PISO: los bits entran de a uno y quedan disponibles todos juntos.
- Ejemplo: una **placa de red**. Por el cable de red los datos viajan en serie, pero el microprocesador los maneja en un bus paralelo. Se necesitan registros para serializar lo que sale y paralelizar lo que entra.

SISO y PIPO (misma forma de entrada y de salida): buffers

- Sirven para **adaptar diferencias de velocidad** entre dos dispositivos.
- Ejemplo: un dispositivo rápido transmite mucha información y uno lento no puede procesarla al mismo ritmo. El rápido **escribe en el buffer** y el lento **toma los datos del buffer** cuando puede.

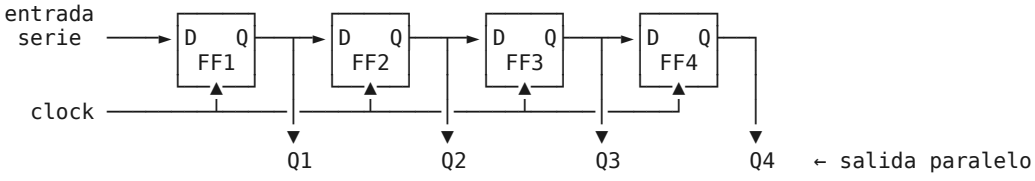
PIPO, uso adicional: capturar datos de un bus

- Las entradas del registro se conectan al bus de datos.
- Un árbitro del bus, un controlador o el propio microprocesador le indica al registro que **saque una foto** de los bits que hay en el bus y los ponga a la salida.
- El hardware que contiene ese registro toma los bits de ahí y los procesa.

12.4 Registro SIPO (4 bits)

Construcción

- 4 flip-flops D uno a continuación del otro.
- **Todos los clocks están unidos** a una misma línea de clock.
- La salida `Q` de cada flip-flop se conecta a la entrada `D` del siguiente.
- La entrada serie es la `D` del primer flip-flop.
- La salida paralelo son las cuatro `Q`.



Funcionamiento

- En cada disparo de clock, el bit de cada entrada `D` pasa a su salida `Q`. Como están encadenados, **todos los bits se desplazan una posición**.
- Si se cambia la entrada sin dar un pulso de clock, no pasa nada (el D no copia hasta que hay disparo).

Ejemplo: cargar la secuencia 0, 1, 1, 0

Pulso	Bit que entra	Q1	Q2	Q3	Q4
inicio	—	0	0	0	0
1	0	0	0	0	0
2	1	1	0	0	0
3	1	1	1	0	0
4	0	0	1	1	0

- Después de **4 pulsos de clock** hay 4 bits cargados. Un dispositivo conectado a las salidas `Q` puede tomarlos en paralelo.
- Es decir: los bits que entran por la única entrada serie se toman por la salida paralelo **cada cuatro pulsos**.
- Para dejarlo listo para una nueva combinación hay dos opciones:
  - usar el **clear** de los cuatro flip-flops, o
  - dejar la entrada serie en 0 y dar más pulsos de clock hasta que los bits terminen de salir.

12.5 Registro SISO (4 bits)

Construcción

- Es **el mismo circuito que el SIPO**. La única diferencia es que se usa solamente la salida `Q` del **último** flip-flop como salida serie.

Funcionamiento (ejemplo con los bits 1 0 1 1)

1. Se cargan los 4 bits por la entrada serie, con 4 pulsos de clock.
2. Al terminar la carga, el **primer bit que entró ya aparece en la salida** (está en el último flip-flop).
3. Con cada pulso siguiente aparece en la salida el bit que venía detrás.
4. Dejando la entrada en 0, después de salir el último bit el registro queda vacío.

En resumen: 4 bits entran por la entrada serie y, con los siguientes pulsos de clock, esos mismos 4 bits salen en el mismo orden por la salida serie.

12.6 Registro PIPO (4 bits)

Construcción

- 4 flip-flops D **independientes entre sí** (no están encadenados).
- Cada `D` es una entrada distinta, para un bit distinto.
- Cada `Q` es una salida independiente.
- Todos los clocks están unidos a la misma entrada de clock (en el video, flanco ascendente).

Funcionamiento (ejemplo con 1 0 1 1)

1. Se colocan los bits `1 0 1 1` en las entradas.
2. Llega el disparo de clock → los 4 bits aparecen **copiados en las salidas**.
3. Si después se modifican las entradas (todas a 0 o todas a 1), las salidas **no cambian**: el registro sigue sosteniendo el valor.
4. Recién con el **siguiente pulso de clock** las salidas toman los nuevos valores de las entradas.

Es el comportamiento de "foto" del flip-flop D, aplicado a varios bits a la vez.

12.7 Registro PISO (4 bits)

Construcción

- 4 flip-flops D.
- Delante de la entrada `D` de cada flip-flop hay un **multiplexor de 2 entradas y 1 salida**.
  - Funciona como una **llave** que elige entre la entrada de arriba (la "0") y la de abajo (la "1").

    - Tiene una entrada de **control**: con control = 0 pasa la entrada 0; con control = 1 pasa la entrada 1.
- El multiplexor permite elegir qué se carga en cada flip-flop:
    - el bit de la **entrada paralelo**, o
    - la salida `Q` del **flip-flop anterior**.
- Todos los controles están unidos a un solo bit: **carga/desplazamiento** (0 = carga, 1 = desplazamiento).
- En modo desplazamiento, el primer flip-flop recibe un **0 fijo**.
- La salida serie es la `Q` del último flip-flop.

Los dos modos

Bit de control	Modo	Qué hace cada flip-flop al llegar el clock
0	**Carga**	Toma el bit de su entrada paralelo
1	**Desplazamiento**	Toma la `Q` del flip-flop anterior (el primero toma un 0)

Proceso completo (ejemplo cargando 1 0 1 0)

1. Control = 0 (modo carga). Se colocan los bits `1 0 1 0` en las entradas paralelo.
2. Pulso de clock → los 4 bits quedan guardados en los 4 flip-flops.
3. Control = 1 (modo desplazamiento).
4. Cada pulso de clock desplaza los bits una posición hacia la salida y **entra un 0 por la izquierda**:

Pulso	FF1	FF2	FF3	FF4 (salida serie)
carga	1	0	1	0
1	0	1	0	1
2	0	0	1	0
3	0	0	0	1
4	0	0	0	0

- Después de 4 pulsos, los bits "fluyeron" hacia la salida de a uno. Analogía del video: como tirar la cadena.

12.8 Cuadro resumen de los registros

Registro	Entrada	Salida	Construcción	Función principal
SIPO	Serie	Paralelo	FF D encadenados; se usan todas las `Q`	Paralelizar datos que llegan en serie
SISO	Serie	Serie	Igual que el SIPO; se usa solo la última `Q`	Buffer / retardo
PIPO	Paralelo	Paralelo	FF D independientes con clock común	Buffer; capturar el valor de un bus
PISO	Paralelo	Serie	FF D + multiplexor en cada entrada (carga/ desplazamiento)	Serializar datos paralelos

13. Ideas clave para repasar

- **Secuencial** = la salida futura depende de las entradas **y** del estado presente. Se logra **realimentando** salidas hacia las entradas.
- **Sincrónico** = con clock, cambia solo cuando el clock dispara. **Asincrónico** = sin clock, cambia apenas cambian las entradas.
- **Clock**: señal periódica. `f = 1 / T`. El período se mide en segundos y la frecuencia en hertz.
- Más frecuencia → pasos más rápidos, pero **más consumo y más calor**. No sirve para comparar micros distintos entre sí.
- **Disparo**: por nivel (alto o bajo) o por flanco (ascendente o descendente). El de flanco es preferible. Los flancos reales son rampas, no verticales.
- **Flip-flop** = memoria de 1 bit, mientras tenga alimentación.
- **SR**: memoria / reset / set / prohibido. **D**: copia D (foto). **T**: memoria / invierte. **JK**: memoria / reset / set / invierte.
- **S, R, D, T, J, K** son sincrónicas (esperan al clock). **Preset y clear** son asincrónicas (no lo esperan) y fijan el estado inicial.
- **Serie** = un bit detrás de otro por una línea. **Paralelo** = todos a la vez, un cable por bit.
- **Registros de desplazamiento** = conjuntos de flip-flops: SIPO, SISO, PIPO, PISO.